

Trabajo Práctico Final — Práctico 9

Testing de Software 2024

Tomás Rodeghiero

Índice

Introducción	2
Ejercicio 1.a: completar la implementación	3
Errores detectados y correcciones aplicadas	3
Convenciones de código aplicadas	8
Ejercicio 1.b: suite de tests para ThreesBoard	8
Técnica seleccionada	8
Características y particiones	8
Criterio de cobertura	9
Requisitos de test	9
Construcción de la suite	10
Requisitos inviables y justificación	11
Cobertura con JaCoCo	11
Ejercicio 2: tests de módulo/integración para los movimientos	12
Objetivo	12
Modelo de estados por acción	12
Inyección de aleatoriedad determinista	13
Requisitos de test	13
Construcción de la suite	14
Requisitos inviables	15
Cobertura con JaCoCo	15
Resultado global	16
Conclusiones	16

Introducción

Este documento corresponde a la resolución del *Trabajo Práctico Final* (Práctico 9) de Testing de Software.

Clases provistas.

- `ThreesBoard.java`: modela el tablero de 4×4 y determina qué movimientos están habilitados en un estado dado.
- `ThreesTile.java`: modela una pieza individual. Una pieza puede ser “libre” (valor 0) o tener un valor válido: 1, 2 o $3 \cdot 2^i$ con $i \in \mathbb{N}$ (es decir, 3, 6, 12, 24, 48, ...).
- `ThreesController.java`: ejecuta los movimientos (`moveUp`, `moveDown`, `moveLeft`, `moveRight`), gestiona la carga de la siguiente baldosa en el borde opuesto al movimiento y conserva `nextTileValue`.
- `RandomGenerator.java` y `RandomValueGenerator.java`: la interfaz y su implementación concreta para la fuente de aleatoriedad. Aproveché esta interfaz en los tests para inyectar una secuencia determinista (`SequenceRandomGenerator`) y de esa forma poder formular aserciones exactas.

Reglas del juego. Las repaso brevemente porque me sirvieron como marco para definir los casos de prueba:

- El tablero es 4×4 y cada partida inicia con 9 piezas aleatorias cuyos valores pertenecen a $\{1, 2, 3\}$.
- Cada movimiento desplaza *todas* las piezas en la misma dirección, a lo sumo un lugar.
- Las **combinaciones** permitidas son: $1 + 2 \rightarrow 3$ (y $2 + 1 \rightarrow 3$), y $x + x \rightarrow 2x$ para $x \geq 3$. Conviene aclarar que $3 + 6$ o $6 + 12$ *no* son combinaciones válidas, aunque a primera vista podría suponerse lo contrario.
- Por fila o columna se realiza *a lo sumo una* combinación por movimiento. Este detalle resultó fundamental al momento de implementar el algoritmo, ya que obliga a recorrer cada línea con cuidado.
- Después de cada movimiento se introduce una baldosa nueva (con valor 1, 2 o 3) en alguna posición del borde opuesto a las filas o columnas que efectivamente se modificaron.
- La partida finaliza cuando no resta ningún movimiento posible.
- **Invariante.** Toda pieza con valor $n \notin \{1, 2\}$ cumple $\exists i \in \mathbb{N} : n = 3 \cdot 2^i$. Este invariante me sirvió para validar tanto la inicialización como la generación aleatoria.

Organización del documento. El práctico está dividido en dos partes, que en este informe organizo de la siguiente manera:

- En el **Ejercicio 1.a** relato la parte correspondiente a completar la implementación: qué defectos detecté, cómo los corregí y cuál fue la justificación de cada cambio.
- En el **Ejercicio 1.b** presento la suite de tests para **ThreesBoard** construida mediante *Caracterización del Dominio de Inputs*, con sus requisitos y la forma en que quedaron cubiertos.
- En el **Ejercicio 2** explico la suite de tests de módulo/integración para **moveUp**, **moveDown**, **moveLeft** y **moveRight**, orientada a alcanzar una alta cobertura de predicados, medida con JaCoCo.

Ejecución del proyecto. Desde `tp9/assignment-9-rodeghiero-1`, ambas suites se ejecutan con un único comando:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q test
```

Ejercicio 1.a: completar la implementación

Errores detectados y correcciones aplicadas

Al comenzar a leer el código provisto identifiqué varios puntos pendientes: métodos sin implementar, validaciones incorrectas y algunos defectos más sutiles. A continuación detallo cada uno, explicando qué encontré y de qué manera lo resolví.

(1) **ThreesBoard.getTile**: la columna podía salirse de rango

El primer problema que detecté fue la guarda de **getTile**. La condición original era `col <= COLUMNS`, lo cual permitía el índice `col == COLUMNS`, que se encuentra fuera de rango. La reemplacé por una comparación estricta:

```
public ThreesTile getTile(int row, int col) {
    if (0 <= row && row < ROWS && 0 <= col && col < COLUMNS) {
        return elements[row][col];
    } else {
        throw new IllegalArgumentException();
    }
}
```

Aproveché la corrección para alinear la guarda con la convención habitual de Java (`<length`) y mantuve el resto del método sin cambios.

(2) ThreesBoard(int totalTiles): sin validación de rango

El constructor que inicializa una cantidad determinada de piezas no validaba su parámetro. Si se invocaba con `totalTiles > 16`, el `while` interno entraba en un bucle infinito, dado que jamás encontraba celdas libres para continuar. Lo corregí incorporando una verificación al inicio del método, siguiendo el principio *fail-fast*:

```
public ThreesBoard(int totalTiles) {
    if (totalTiles < 0 || totalTiles > ROWS * COLUMNS) {
        throw new IllegalArgumentException("totalTiles must be in
            [0, 16]");
    }
    // ... resto de la inicialización ...
}
```

(3) ThreesBoard: valor aleatorio inicial mal elegido

La inicialización aleatoria asignaba valores entre 0 y 2, **incluyendo el 0**. El inconveniente es que 0 representa una celda libre, de modo que el constructor incurría en una contradicción: pretendía colocar una pieza pero, en ocasiones, dejaba la celda vacía. Lo modifiqué a `generator.getRandom(3) + 1`, garantizando así un valor en el rango [1, 3], que es el rango correspondiente a las baldosas iniciales del juego.

(4) ThreesBoard: numerosos métodos sin implementar

Varios métodos centrales se encontraban directamente vacíos:

- `isFinished()`
- `canMoveLeft()`, `canMoveRight()`, `canMoveUp()`, `canMoveDown()`
- `canTilesCombine(ThreesTile, ThreesTile)`
- `computeScore()`

Con la intención de evitar duplicación de lógica, factoricé las decisiones comunes en `canTilesCombine`, de modo tal que todos los `canMove*` pudieran delegar en esa función la verificación de combinabilidad entre dos piezas. `isFinished()` quedó expresado como la negación conjunta de los cuatro `canMove*`.

Para cada `canMove*` la lógica es análoga: recorrer el tablero en la dirección del movimiento y devolver `true` apenas se identifique al menos una celda no libre que pueda avanzar (ya sea porque su vecina en esa dirección está libre, o porque ambas piezas son combinables).

```
public boolean canMoveLeft() {
```

```

    for (int i = 0; i < ROWS; i++) {
        for (int j = 1; j < COLUMNS; j++) {
            if (!elements[i][j].isFree()) {
                ThreesTile current = elements[i][j];
                ThreesTile left = elements[i][j - 1];
                if (left.isFree() || canTilesCombine(left,
                    current)) {
                    return true;
                }
            }
        }
    }
    return false;
}

public boolean canTilesCombine(ThreesTile t1, ThreesTile t2) {
    if (t1 == null || t2 == null) {
        throw new IllegalArgumentException("Tiles cannot be null"
            );
    }
    if (t1.isFree() || t2.isFree()) return false;
    int v1 = t1.getValue(), v2 = t2.getValue();
    return (v1 == 1 && v2 == 2)
        || (v1 == 2 && v2 == 1)
        || (v1 >= 3 && v1 == v2);
}

```

Los tres `canMove*` restantes son simétricos, modificando únicamente los índices del recorrido.

Respecto a `computeScore`, traduzca la regla de puntaje del juego como una sumatoria sobre todas las celdas del tablero:

```

public int computeScore() {
    int score = 0;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLUMNS; j++) {
            int v = elements[i][j].getValue();
            if (v == 3) score += 3;
            else if (v > 3) score += (v - 3) * 3;
        }
    }
    return score;
}

```

```
}
```

(5) `ThreesTile.isValidValue`: validación correcta del invariante

En este punto debí implementar la validación del invariante de la pieza. Un valor v es válido si se trata de uno de los casos base (0, 1 o 2) o si puede escribirse como $3 \cdot 2^i$ para algún $i \geq 0$. Para simplificar la verificación, reduje el problema a comprobar que $v/3$ sea potencia de dos:

```
public boolean isValidValue(int v) {
    if (v == 0 || v == 1 || v == 2) return true;
    if (v < 3 || v % 3 != 0)         return false;
    int quotient = v / 3;
    return isPowerOfTwo(quotient);
}

private boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Recurrí al truco clásico $n \& (n - 1) == 0$ para identificar potencias de dos: es una única operación a nivel de bits y deja el chequeo bastante limpio.

(6) `ThreesTile.setRandomValue`: distribución incorrecta

El código original ejecutaba `value = randomGenerator.nextInt() % 3`, lo cual podía producir valores fuera de $\{1, 2, 3\}$ (incluso negativos, dado que `nextInt()` sin cota devuelve cualquier entero). Lo corregí mediante `value = randomGenerator.nextInt(3) + 1`, que garantiza el rango exacto.

(7) `ThreesController`: tres movimientos faltantes y un cuarto con defecto

Esta fue la parte más extensa del ejercicio. `moveUp`, `moveLeft` y `moveRight` no se encontraban implementados, y `moveDown` presentaba un problema serio: podía duplicar valores durante el desplazamiento, ya que no respetaba la regla de *una sola* combinación por fila o columna.

En lugar de duplicar el código de `moveDown` corregido para cada dirección, opté por factorizar la lógica en *helpers* privados, de modo que cada dirección quedara como una variante reducida del mismo flujo. Todos los *helpers* cuentan con su `JavaDoc` correspondiente, describiendo su propósito y sus precondiciones y postcondiciones:

- `readRow` y `readColumn`: leen una línea del tablero respetando el orden del movimiento.

- `collapseAndMergeLine`: colapsa los huecos y aplica, a lo sumo, una combinación por par adyacente.
- `sameLine`: compara dos líneas para determinar si el movimiento produjo cambios efectivos.
- `writeRow` y `writeColumn`: escriben nuevamente los valores en las coordenadas del tablero.
- `loadNextTileOnColumns` y `loadNextTileOnRows`: cargan la baldosa nueva en el borde opuesto a las filas/columnas que se modificaron.

El núcleo del algoritmo es `collapseAndMergeLine`. Lo escribí siguiendo dos pasos: primero compactar (eliminar los 0 intermedios) y luego recorrer la línea de izquierda a derecha combinando de a pares cuando corresponde:

```
private int[] collapseAndMergeLine(int[] line) {
    int[] compact = new int[line.length];
    int n = 0;
    for (int v : line) if (v != 0) compact[n++] = v;

    int[] merged = new int[line.length];
    int m = 0, i = 0;
    while (i < n) {
        if (i + 1 < n && canValuesCombine(compact[i], compact[i + 1])) {
            merged[m++] = compact[i] + compact[i + 1];
            i += 2;
        } else {
            merged[m++] = compact[i];
            i++;
        }
    }
    return merged;
}
```

La clave para garantizar la regla de “una sola combinación por línea” radica en que, cuando se combinan dos elementos, el índice `i` se incrementa en dos. De esta forma, luego de fusionar `compact[i]` y `compact[i+1]`, el siguiente par a evaluar es `(i+2, i+3)` y no `(i+1, i+2)`, evitando así reutilizar un elemento ya combinado.

(8) pom.xml: incompatibilidad de JaCoCo con JDK 17

Al intentar ejecutar la cobertura, el plugin de JaCoCo (0.8.2) fallaba al instrumentar bytecode generado por JDK 17, ya que no soportaba versiones tan recientes. Lo actualicé a 0.8.12 y a partir de ese momento la ejecución resultó exitosa.

Convenciones de código aplicadas

Con el objetivo de mantener el código prolijo y consistente, seguí las siguientes convenciones:

- Todos los métodos que agregué incluyen su correspondiente **JavaDoc**, indicando parámetros, valores de retorno y excepciones.
- Las validaciones de entrada se ubican al inicio del método (*fail-fast*) y lanzan `IllegalArgumentException` con un mensaje informativo.
- Utilicé las constantes `ThreesBoard.ROWS` y `ThreesBoard.COLUMNS` en lugar de *magic numbers* dispersos por el código.
- Toda la lógica de aleatoriedad se canaliza a través de `RandomGenerator`, lo cual me permitió inyectar generadores deterministas en los tests sin necesidad de modificar el código de producción.

Ejercicio 1.b: suite de tests para ThreesBoard

Técnica seleccionada

Para construir la suite empleé **Caracterización del Dominio de Inputs**. Me pareció la opción más natural, ya que la API de `ThreesBoard` expone métodos con dominios discretos bastante claros (coordenadas, cantidades, tipos de pareja, entre otros) y porque la consigna habilitaba dejar fuera `toString()` y `getRandom()`, lo cual contribuye a acotar el modelo.

Características y particiones

A continuación detallo las características que identifiqué y la forma en que las dividí en bloques:

ID	Característica	Particiones (bloques)
C1	coordenadas (<code>setTile/getTile</code>)	válidas [0..3]; inválidas (< 0 o ≥ 4)
C2	cantidad inicial (<code>ThreesBoard(totalTiles)</code>)	válidas [0..16]; inválidas (< 0 o > 16)
C3	ocupación (<code>numberOfSetTiles</code>)	vacío; parcial; completo
C4	tipo de pareja (<code>canTilesCombine</code>)	$1 + 2$; $2 + 1$; $x + x$ con $x \geq 3$; no combinables; <code>null</code>
C5	movilidad (<code>canMove*</code> , <code>isFinished</code>)	sin movs; por huecos; por combinación; lleno bloqueado
C6	categoría de valor (<code>computeScore</code>)	$\{0, 1, 2\}$; 3 ; > 3

Criterio de cobertura

Apliqué **Each Choice Coverage**, extendido a representantes razonables de cada bloque. Para las características que presentan interacción natural con la dirección (como `canMove` en cada una de las cuatro direcciones), me aseguré de ejercitar cada partición en las cuatro direcciones y no únicamente en una.

Requisitos de test

A partir de los bloques definidos para cada característica, formulé los siguientes requisitos:

- **R1.** `setTile/getTile` dentro del rango actualizan y recuperan correctamente el valor de la celda. (*C1, bloque válido.*)
- **R2.** `setTile` rechaza coordenadas fuera de rango. (*C1, bloque inválido.*)
- **R3.** `getTile` rechaza coordenadas fuera de rango. (*C1, bloque inválido.*)
- **R4.** `ThreesBoard(totalTiles)` con un valor válido inicializa exactamente esa cantidad de celdas, con valores permitidos. (*C2 válido + C3 parcial.*)
- **R5.** `ThreesBoard(totalTiles)` rechaza un `totalTiles` inválido. (*C2 inválido.*)
- **R6.** `numberOfSetTiles` cuenta exactamente las celdas no vacías. (*C3.*)
- **R7.** `canTilesCombine` respeta las reglas tanto en los casos válidos como en los inválidos, y rechaza `null`. (*C4.*)

- **R8.** Sobre un tablero vacío, ningún movimiento resulta posible y queda finalizado. (*C5 “sin movs”.*)
- **R9.** Sobre un tablero con huecos, el movimiento por desplazamiento queda detectado en cada dirección. (*C5 “por huecos”.*)
- **R10.** Sobre un tablero con piezas combinables, cada `canMove*` detecta la combinación. (*C5 “por combinación”.*)
- **R11.** Un tablero lleno y sin combinaciones queda finalizado. (*C5 “lleno bloqueado” + C3 “completo”.*)
- **R12.** `computeScore` suma 3 por cada celda con valor 3 y $(v - 3) \cdot 3$ por cada celda con $v > 3$. (*C6.*)

Construcción de la suite

La suite se denomina `ThreesBoardInputDomainCharacterizationTest`. Las decisiones de implementación más relevantes son las siguientes:

- Empléé `@BeforeEach` para crear un tablero limpio en cada test y garantizar el aislamiento. Esta práctica reduce considerablemente el *boilerplate* y asegura que los tests sean independientes entre sí.
- Los rechazos (R2, R3, R5) se prestan naturalmente a la parametrización, por lo cual utilicé `@ParameterizedTest` con `@CsvSource`.
- Las reglas de combinación (R7) también quedaron sustancialmente más claras al parametrizarlas, en lugar de incluir un único test extenso con múltiples aserciones.
- Los nombres de los tests son auto-explicativos: leer `setTileRejectsOutOfRangeCoordinates` ya revela toda la intención, sin necesidad de abrir el cuerpo del test.

Mapeo requisito $\rightarrow$ test:

Requisito	Test que lo cubre
R1	<code>setAndGetTileWithinBounds</code>
R2	<code>setTileRejectsOutOfRangeCoordinates</code> (parametrizado)
R3	<code>getTileRejectsOutOfRangeCoordinates</code> (parametrizado)
R4	<code>constructorWithTotalTilesSetsExactAmount</code>
R5	<code>constructorRejectsInvalidTotalTiles</code> (parametrizado)
R6	<code>numberOfSetTilesCountsNonFreeTiles</code>
R7	<code>canTilesCombineMatchesRules</code> + <code>canTilesCombineRejectsNullTiles</code>
R8	<code>emptyBoardHasNoMovesAndIsFinished</code>
R9	<code>movementByGapsIsDetectedPerDirection</code>
R10	<code>movementByCombinationIsDetectedPerDirection</code>
R11	<code>fullBoardWithoutCombinationsIsFinished</code>
R12	<code>computeScoreFollowsImplementedRules</code>

Requisitos inviables y justificación

Existen ciertos requisitos que, si bien podrían parecer válidos en un primer momento, no resulta razonable intentar cubrirlos mediante tests unitarios. Los marqué como inviables y expongo a continuación el motivo correspondiente:

- **RI1.** Verificar las *posiciones exactas* de las fichas asignadas por `ThreesBoard(totalTiles)` sin mockear `RandomGenerator`. Las posiciones dependen del generador aleatorio, de modo que, sin un doble determinista, el test resultaría *flaky*.
- **RI2.** Cobertura exhaustiva de todos los estados posibles del tablero. Con 16 celdas y múltiples valores posibles por celda, el espacio de estados se vuelve inabordable. Justamente por esa razón opté por particionado por dominio de inputs.
- **RI3.** Verificar la uniformidad estadística del generador aleatorio. Dicha propiedad es de naturaleza estadística y no puede validarse con aserciones deterministas en una corrida de tests unitarios.

Cobertura con JaCoCo

Sobre `ThreesBoard` (ejercicio 1.b) se obtuvieron las siguientes métricas:

Métrica	Valor
Líneas	85/97 = 87.63 %
Branches	108/124 = 87.10 %
Métodos	12/13 = 92.31 %

Las porciones que quedaron sin cubrir corresponden, en líneas generales, a los siguientes casos:

- Ramas defensivas que ya resultan ejercitadas indirectamente por la suite (por ejemplo, comparaciones que se cumplen siempre dada la naturaleza del dominio elegido).
- `toString()`: lo excluyo explícitamente del modelo, ya que la consigna lo permitía.
- Algunas ramas del constructor aleatorio que dependen de la fuente de aleatoriedad (cuyo motivo fue desarrollado en RI1).

Ejercicio 2: tests de módulo/integración para los movimientos

Objetivo

En este ejercicio el objetivo difiere del anterior: en lugar de aislar métodos pequeños, me propuse ejercitar las acciones de `ThreesController` (`moveUp`, `moveDown`, `moveLeft`, `moveRight`) en busca de una **alta cobertura de predicados** medida con JaCoCo. La consigna establece que un test que ejecuta *dos movimientos distintos en secuencia* cuenta como test de módulo, de modo que incluyo uno con esas características a fin de cumplir explícitamente con el requisito.

Modelo de estados por acción

A fin de evitar quedarme con una noción difusa de “probar los movimientos”, definí cinco estados representativos por los que puede atravesar cada acción. Esto me proporcionó una guía clara para construir los casos de prueba:

- **M0**: tablero *no* modificable en esa dirección (no se mueve nada y `loadNextTile` ni siquiera se invoca).
- **M1**: tablero modificable *únicamente por desplazamiento* (existen huecos en la dirección del movimiento).
- **M2**: tablero modificable mediante *combinación* $1 + 2$ o $2 + 1$.
- **M3**: tablero modificable mediante *combinación* $x + x$ para $x \geq 3$.
- **M4**: carga de *una nueva baldosa* en el borde opuesto, con fila/columna elegida pseudo-aleatoriamente.

Inyección de aleatoriedad determinista

En este punto aproveché el diseño del código: dado que las acciones de movimiento invocan `generator.getRandom(...)` para seleccionar la posición de la nueva baldosa y para regenerar `nextTileValue`, pude inyectar un `SequenceRandomGenerator` (clase privada del test) que devuelve los enteros de una secuencia preconfigurada. De esta forma pude formular aserciones *exactas* sobre la posición final y el siguiente valor cargado, en lugar de conformarme con aserciones más laxas del estilo “alguna celda del borde derecho”.

```
private static class SequenceRandomGenerator implements
RandomGenerator {
    private final int[] sequence;
    private int index;
    SequenceRandomGenerator(int... sequence) { this.sequence =
        sequence; this.index = 0; }
    @Override public int getRandom(int bound) {
        if (bound <= 0) throw new IllegalArgumentException("bound
            must be positive");
        if (index >= sequence.length) return 0;
        int value = sequence[index++];
        return Math.floorMod(value, bound);
    }
}
```

Requisitos de test

- **R2.1.** El constructor por defecto produce un tablero inicial válido (9 baldosas y `nextTileValue` en $[1, 3]$).
- **R2.2.** Si el tablero no admite cambios en una dirección, el movimiento retorna `false` y no modifica ni el tablero ni `nextTileValue`. Se parametrizó en las cuatro direcciones (estado M0).
- **R2.3.** `moveLeft` combina, desplaza y carga la nueva baldosa en el borde derecho (M1 + M3).
- **R2.4.** `moveRight` ejecuta la combinación 2 + 1 y la carga en el borde izquierdo (M2).
- **R2.5.** `moveUp` combina y carga en la fila inferior (M3).
- **R2.6.** `moveDown` combina y carga en la fila superior (M3 + M4).
- **R2.7.** Test de módulo: dos movimientos distintos en secuencia se comportan de manera consistente (M2/M3 + verificación de estado global).

Construcción de la suite

La suite se denomina `ThreesControllerModuleIntegrationTest`.

Mapeo requisito → test:

Requisito	Test que lo cubre
R2.1	<code>defaultConstructorCreatesExpectedInitialState</code>
R2.2	<code>noChangeMoveKeepsBoardAndNextTile</code> (parametrizado, 4 direcciones)
R2.3	<code>moveLeftCombinesShiftsAndLoadsNextTileOnRightEdge</code>
R2.4	<code>moveRightCombinesTwoPlusOneAndLoadsNextTileOnLeftEdge</code>
R2.5	<code>moveUpCombinesAndLoadsNextTileOnBottomRow</code>
R2.6	<code>moveDownCombinesAndLoadsNextTileOnTopRow</code>
R2.7	<code>sequentialDifferentMovesBehaveConsistently</code>

Ejemplo concreto. A modo ilustrativo, presento el test correspondiente a `moveLeft`:

```
@Test
@DisplayName("moveLeft: combina, desplaza y carga nueva baldosa
en borde derecho")
void moveLeftCombinesShiftsAndLoadsNextTileOnRightEdge() {
    load(board, new int[][] {
        {0, 1, 2, 0},
        {3, 3, 0, 0},
        {6, 12, 0, 0},
        {0, 0, 0, 0}
    });
    board.generator = new SequenceRandomGenerator(
        1, // constructor: nextTileValue = 2
        1, // moveLeft: elige movedRows[1] => fila 1
        0 // nuevo nextTileValue = 1
    );
    ThreesController controller = new ThreesController(board);

    boolean moved = controller.moveLeft();

    assertTrue(moved);
    assertBoardEquals(new int[][] {
        {3, 0, 0, 0}, // 1+2 = 3, queda en columna 0
        {6, 0, 0, 2}, // 3+3 = 6; baldosa nueva (valor 2) en
                     // borde derecho
        {6, 12, 0, 0}, // sin combinación: 6 y 12 no se combinan
        {0, 0, 0, 0}
    });
}
```

```

    }, board);
    assertEquals(1, controller.getNextTileValue());
}

```

Lo destacable de este test es que ejercita tres estados de manera simultánea (M1 con el desplazamiento del 1 hacia el 0, M2 con 1 + 2 y M3 con 3 + 3). Además, la fila 2 funciona como contraejemplo para evidenciar que 6 y 12, aunque ambos sean ≥ 3 , no se combinan, ya que no son iguales.

Requisitos inviables

- **RI2.1.** Comprobar la *distribución estadística* de la baldosa nueva a partir de aleatoriedad real. Esto no resulta razonable como test de integración reproducible; por ese motivo trabajo con `SequenceRandomGenerator` y dejo fuera la propiedad estadística.
- **RI2.2.** Cobertura exhaustiva de todas las secuencias posibles de estados del juego. La explosión combinatoria la torna inviable, por lo cual aplico particionado representativo del dominio (M0–M4).

Cobertura con JaCoCo

Sobre `ThreesController` (ejercicio 2):

Métrica	Valor
Instrucciones	596/596 = 100.00 %
Branches	71/78 = 91.03 %
Líneas	124/124 = 100.00 %
Métodos	19/19 = 100.00 %

Las 7 branches que quedaron sin cubrir (de un total de 78) corresponden a casos defensivos, tales como:

- Comprobaciones simétricas en `loadNextTileOnColumns` y `loadNextTileOnRows` para movimientos sin cambios (cuando `movedColumns` o `movedRows` resultan vacíos), que ya quedan cubiertos por el flujo principal.
- Algún camino interno de `canValuesCombine` alcanzable únicamente con combinaciones que no aparecen en las configuraciones representativas elegidas (por ejemplo, $a = 1 \wedge b \neq 2$ con $b = 0$).

Opté por no inflar la suite con tests artificiales destinados exclusivamente a forzar la cobertura de esas ramas, dado que no aportarían capacidad de detección de defectos

reales. La consigna solicita *justificar*, por lo cual prefiero ser explícito sobre la razón de su presencia.

Resultado global

Al ejecutar la build completa:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q test
```

se obtiene:

- Total de tests ejecutados: **38** (entre ambas suites).
- Fallos: **0**.
- Errores: **0**.

Conclusiones

En retrospectiva, considero que este trabajo cierra de forma muy correcta el ciclo completo de la materia:

- El práctico me llevó a *leer código ajeno*, *completar la implementación* (detectando y corrigiendo defectos), *modelar* el espacio de pruebas con una técnica formal (caracterización del dominio de inputs) y *verificar* la suite con cobertura medida por una herramienta externa (JaCoCo). Cada paso involucra contenidos diferentes vistos a lo largo del cuatrimestre.
- La separación entre **ThreesBoard** (modelo y reglas) y **ThreesController** (acciones) facilitó que la división del práctico en dos niveles resultara natural: tests *unitarios* sobre el tablero y tests *de módulo/integración* sobre el controlador.
- La inyección de **RandomGenerator** como dependencia fue lo que me permitió formular aserciones exactas sobre acciones internamente aleatorias. De no haberla contado, varios requisitos hubieran ido a parar a la lista de inviables.
- Los requisitos clasificados como inviables (RI1–RI3 y RI2.1–RI2.2) los dejé explícitos de manera deliberada. Resulta más honesto exponer los límites del enfoque determinista que ocultarlos: existen aspectos que un test unitario o de integración no puede verificar de forma razonable, y separarlos claramente del resto otorga mayor peso a aquello que sí queda cubierto.